

Assignment 4: Improving Requirement Reliability in a Multi-Agent Social Media System

Objective

This assignment improves the multi-agent social media system developed and evaluated in the earlier assignments. Assignment 3 showed that a multi-agent pipeline is easier to inspect than a single-agent pipeline, but orchestration alone does not guarantee that user requirements are preserved. If the system extracts the wrong topic or loses a visual constraint, later agents can complete the workflow and still produce an unsuitable post.

The research question for this assignment is:

Can a structured request-verification stage and a requirement-aware QA loop improve the reliability of a multi-agent social media generation pipeline?

Code repository link: <https://github.com/SelenArikan/agentlc>

Part 1 – Baseline System

Problem Definition

The system automates social media post preparation. A user provides a prompt such as "Make a post about Pilates" or a more detailed product request. The pipeline should:

- Infer the post topic and language.
- Decide whether writing, visual generation, QA, and upload are needed.
- Produce a caption.
- Produce a visual brief and image file when media is requested.
- Validate the output.
- Simulate a local browser upload when publishing is requested.

Baseline Agent Roles

Agent	Baseline Role
Task Manager	Creates route, validates steps, retries failed agents
Trend Researcher	Adds keywords and hashtags for short or trend-based prompts
Content Writer	Writes caption and rough media prompt
Visual Prompt Engineer	Converts rough media prompt into structured photo prompt
Media Creator	Creates an image with provider or local placeholder path
QA Agent	Checks caption/media completeness and safety
Browser Operator	Simulates upload to local demo page

Baseline Workflow vs Improved Pipeline

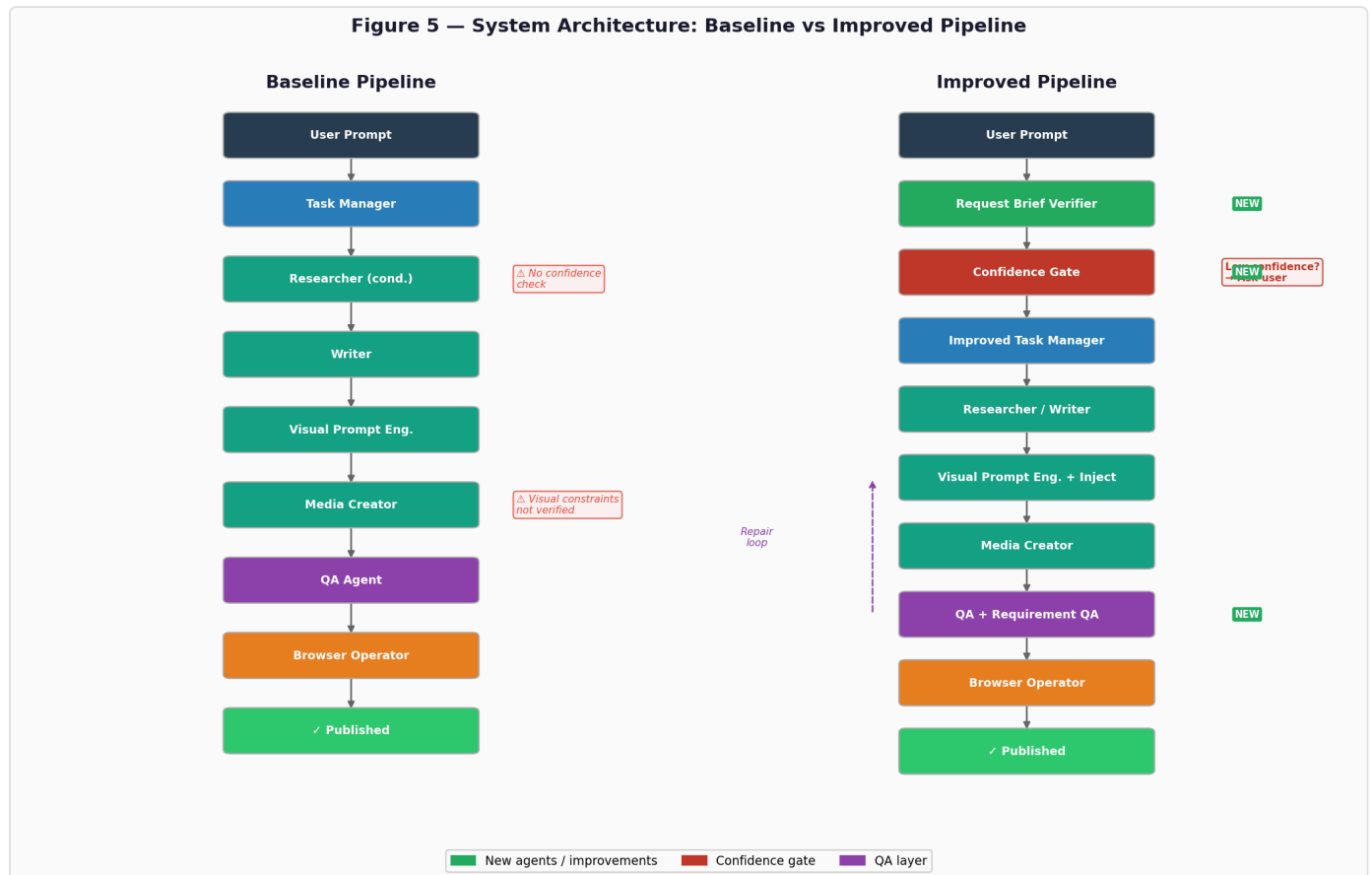


Figure 5 — System Architecture: Baseline Pipeline (left) vs Improved Pipeline with new verification stages (right)

Current Limitations of the Baseline

- A prompt containing an ambiguous reference can produce a generic topic such as *this* and *it*.
- A content pipeline can approve a post although a specific image instruction is only partially represented.
- Completion success can hide upstream mistakes because every downstream agent works from the initial topic.
- More agents produce more external model/API calls, increasing timeout and provider-failure exposure.

Part 2 – Identify Weaknesses

Weakness 1: Low-Confidence Topic Extraction

The baseline Task Manager tries to infer a topic from prompt text. For vague prompts such as "Make a post about this and share it," the baseline may extract the literal phrase *this* and *it* as a topic. The remaining agents then receive that topic and produce a generic caption and generic image brief.

Why it occurs:

- The pipeline treats any non-empty topic-like phrase as usable unless it matches a small blocked set.
- The baseline does not maintain an explicit confidence score for topic extraction.
- QA sees a caption and media file, so it can approve the run even though the topic is semantically weak.

Why it matters: The system can publish a meaningless post. Later agents cannot recover if the initial topic is wrong.

Weakness 2: User Visual Requirements Are Not Explicit Contracts

Some prompts contain visual constraints such as "Outdoor mat Pilates in a park, not an indoor studio" or "A close-up of a woman's ear with three silver earrings." The baseline QA Agent mainly checks safety and media existence, not every visual condition as an explicit checklist.

Why it occurs: Visual constraints are carried in free-form text fields. A media prompt can mention the topic *silver earrings* while missing details such as count or close-up framing.

Why it matters: A generated post can be plausible but not satisfy the user request. Product posts are especially sensitive to count, material, setting, and composition constraints.

Weakness 3: Completion Does Not Mean Reliability

Multi-agent orchestration adds logs, validation calls, and provider calls. In real API mode, Assignment 3 exposed timeouts and provider connection errors. Extra agent calls increase the surface area for latency and service failures.

Why it occurs: Specialized agents create more handoffs than a single combined call. Text and image providers can fail independently.

Why it matters: The system needs verification and fallback behavior, not only more agents. Some steps should stop early when confidence is low.

Part 3 – System Improvements

Three connected improvements were implemented, designed to improve requirement quality before adding more generation cost.

Improvement 1: Structured Request Brief Verifier

Added RequestBriefVerifierAgent in agents/requirement_brief.py. Before the improved Task Manager executes the normal route, this verifier creates a structured brief:

```
{
  "topic": "silver earrings",
  "language": "en",
  "media_required": true,
  "visual_constraints": [
    "close-up composition",
    "earrings visible on a woman's ear",
    "three earrings",
    "silver earrings"
  ],
  "avoid_constraints": [],
  "confidence": 0.92
}
```

Improvement 2: Confidence Gate and Human Approval State

The improved pipeline uses ImprovedTaskManagerAgent in agents/improved_task_manager.py. If the request brief confidence is low and the topic is ambiguous, the improved system does not proceed blindly — it asks for topic clarification. This is a controlled human-in-the-loop change:

- High-confidence prompts continue automatically.
- Ambiguous prompts ask for clarification.
- Repaired or low-confidence publish operations can require human approval.

Improvement 3: Requirement-Aware QA and Feedback Repair

Added RequirementAwareQAAgent in agents/requirement_qa.py. It runs after normal QA approval and checks the structured request brief. If this QA rejects a visual requirement mismatch, the improved Task Manager routes the feedback back into the visual branch for repair.

Implementation Footprint

File	Purpose
models.py	Adds request brief, confidence, and human approval state
agents/requirement_brief.py	Structured request brief and constraint injection
agents/requirement_qa.py	Requirement-aware verification
agents/improved_task_manager.py	Improved route, feedback repair, approval gate
tools/run_assignment4_experiment.py	Original vs improved experiment runner
tests/test_assignment4_improvements.py	Focused improvement tests

Part 4 – Experimental Comparison

Experiment Design

The experiment compares original_multi_agent (existing baseline TaskManagerAgent) with improved_multi_agent (ImprovedTaskManagerAgent). Both systems use the same seven prompts in deterministic local mode (--provider mock --image-provider pillow).

Test Cases

Case	Weakness Target
A4-T1 Short Pilates	No-regression clear prompt
A4-T2 Ambiguous this	Confidence gate and clarification
A4-T3 Outdoor mat Pilates	Preserve environment and mat/studio constraint
A4-T4 Jewelry constraints	Preserve close-up, count, and material
A4-T5 Exact caption	Preserve mixed caption + image intent
A4-T6 Turkish outdoor	Preserve Turkish topic and visual direction
A4-T7 Caption only	Preserve efficient text-only route

Aggregate Results

System	Mean Overall	Correctness	Requirement	Completeness	Latency ms	Logs	LLM Step Proxy	Human Approvals	Failure Rate
Original Multi-Agent	0.85	0.85	0.95	0.86	15.02	13.71	3.00	0.00	0.14
Improved Multi-Agent	1.00	1.00	1.00	1.00	10.42	14.71	2.43	0.14	0.00

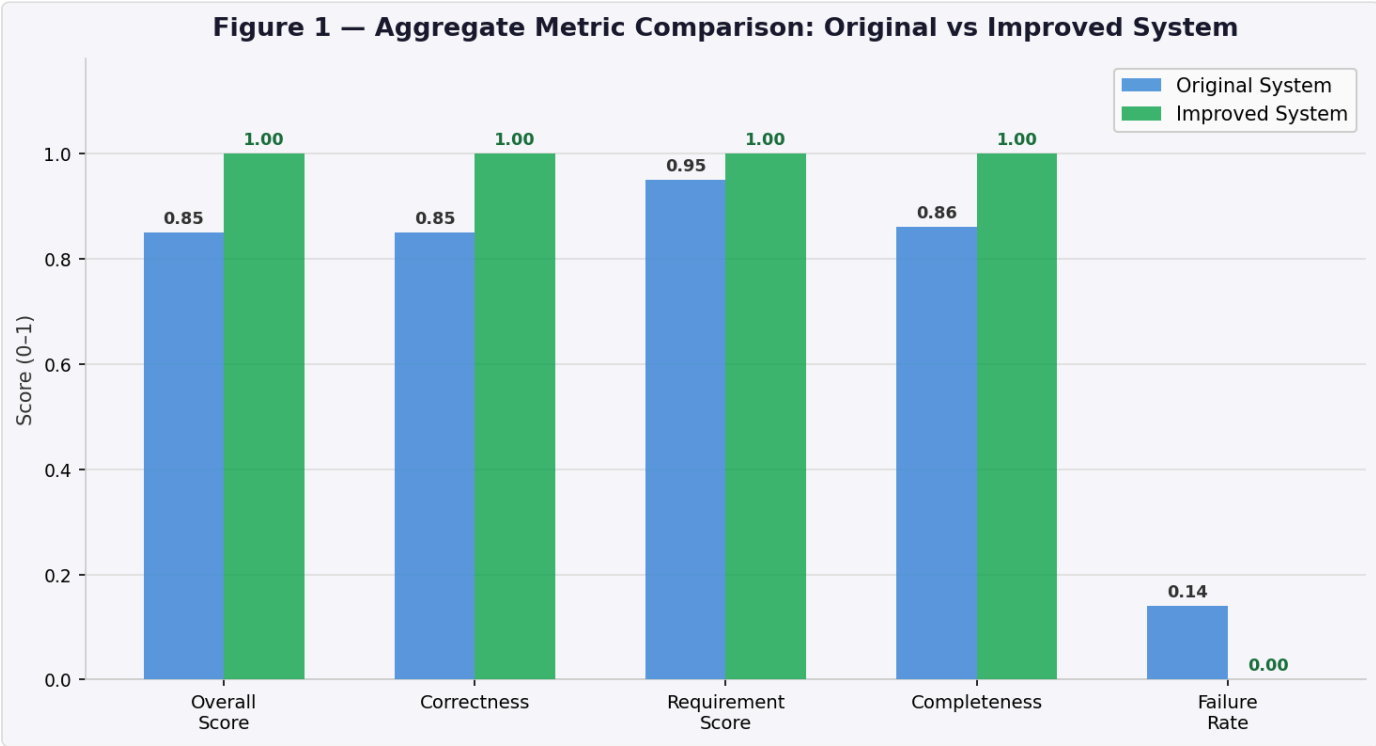


Figure 1 — Aggregate metric comparison across all test cases. The improved system achieves perfect scores on all dimensions and eliminates the failure rate.

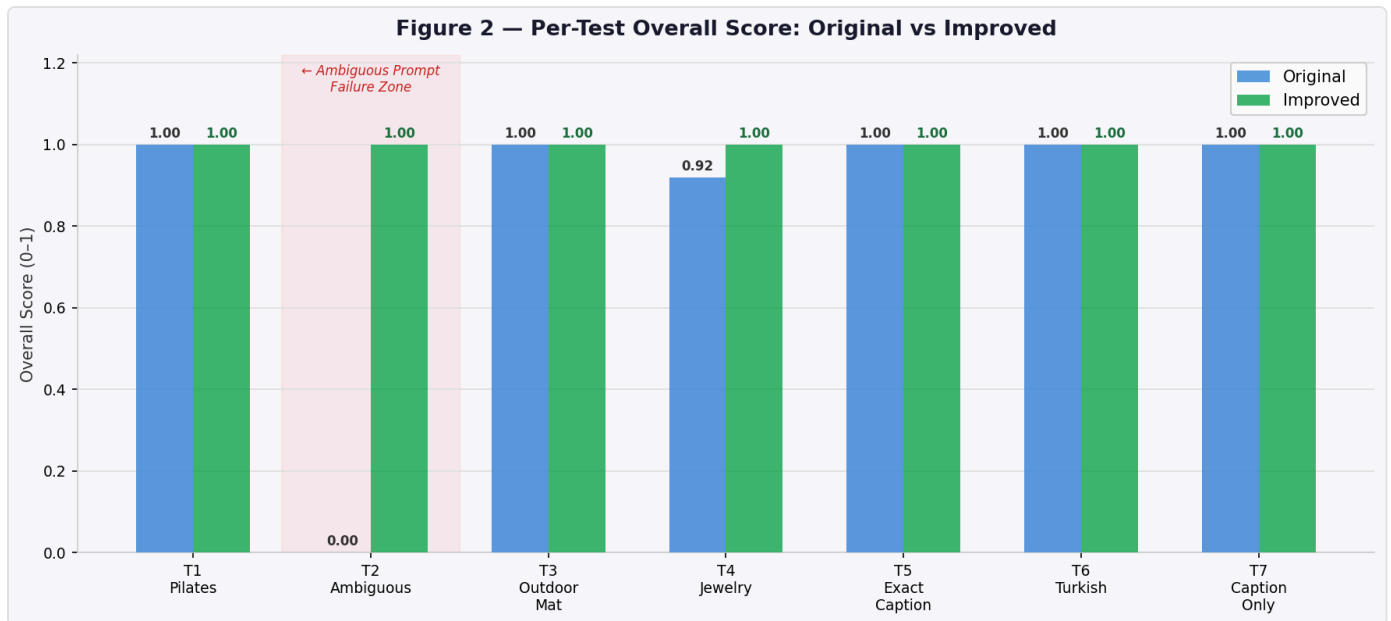


Figure 2 — Per-test overall score. The ambiguous prompt T2 (red zone) exposes the original system's 0.00 score, while the improved system scores 1.00 by requesting clarification instead of publishing.

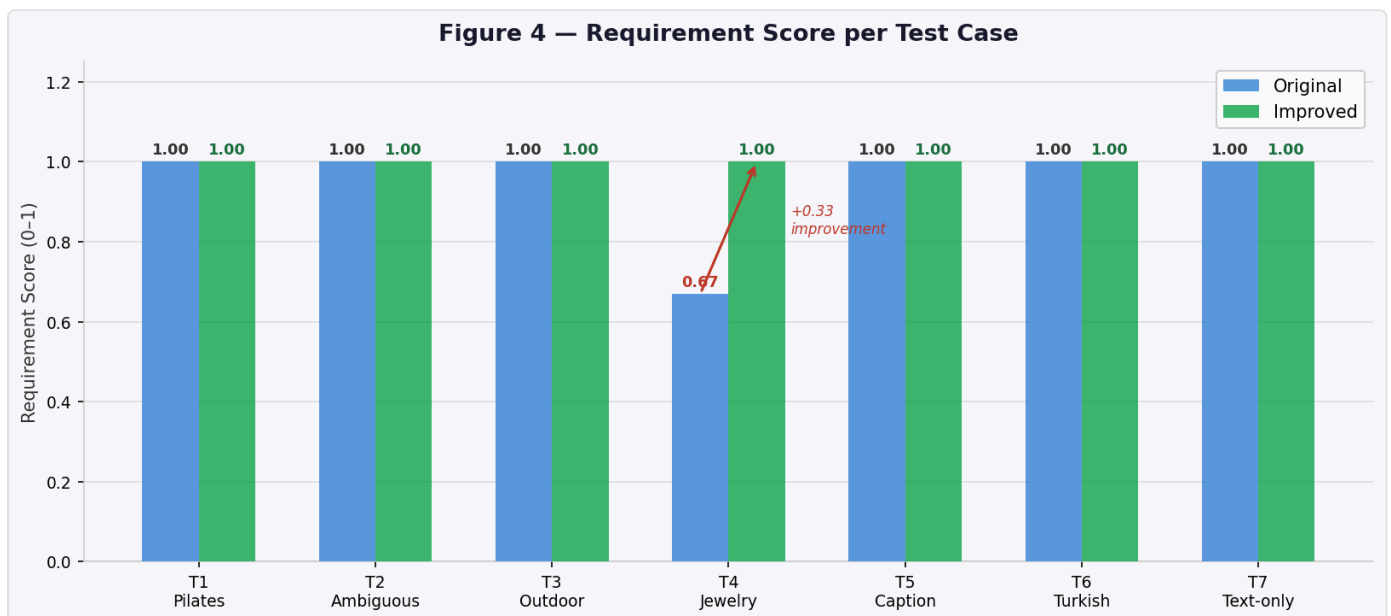


Figure 4 — Requirement score per test case. The jewelry constraint test (T4) shows a +0.33 gain from structured visual constraint injection.

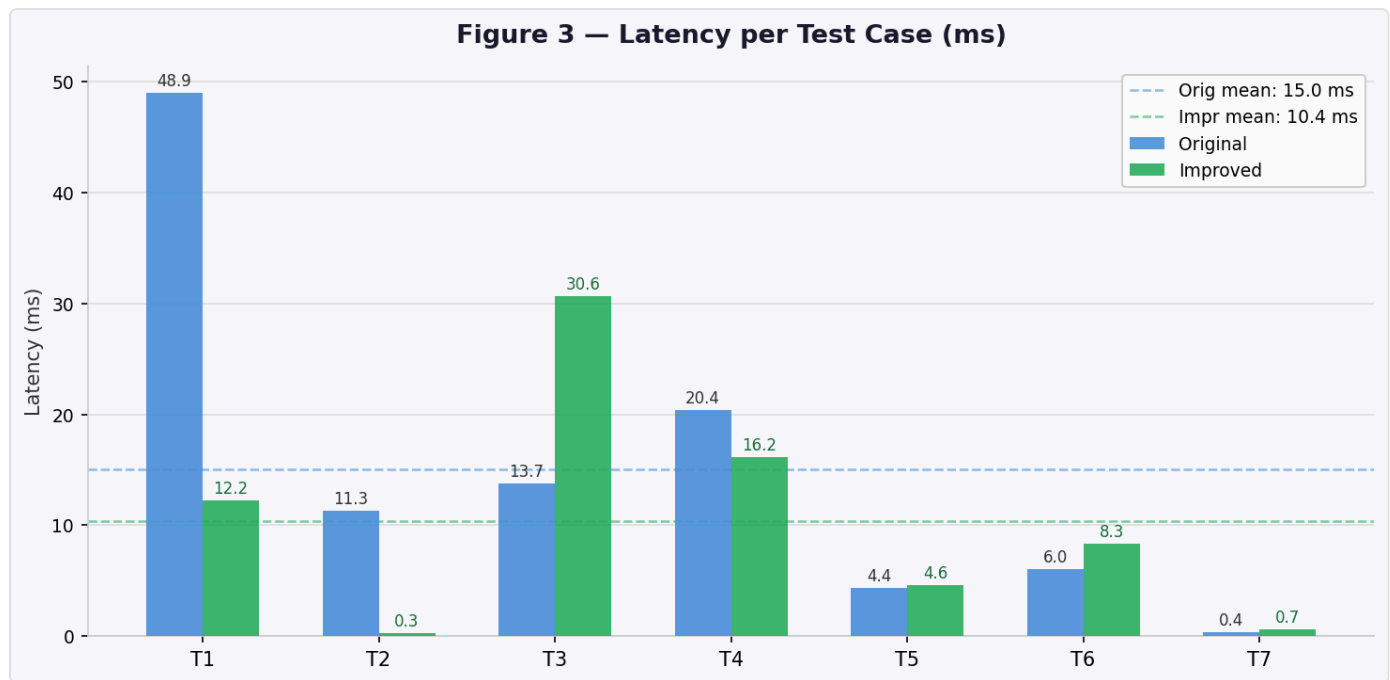


Figure 3 — Latency per test case (ms). The improved system achieves lower mean latency (10.4 ms vs 15.0 ms) by stopping early on low-confidence prompts instead of running the full pipeline.

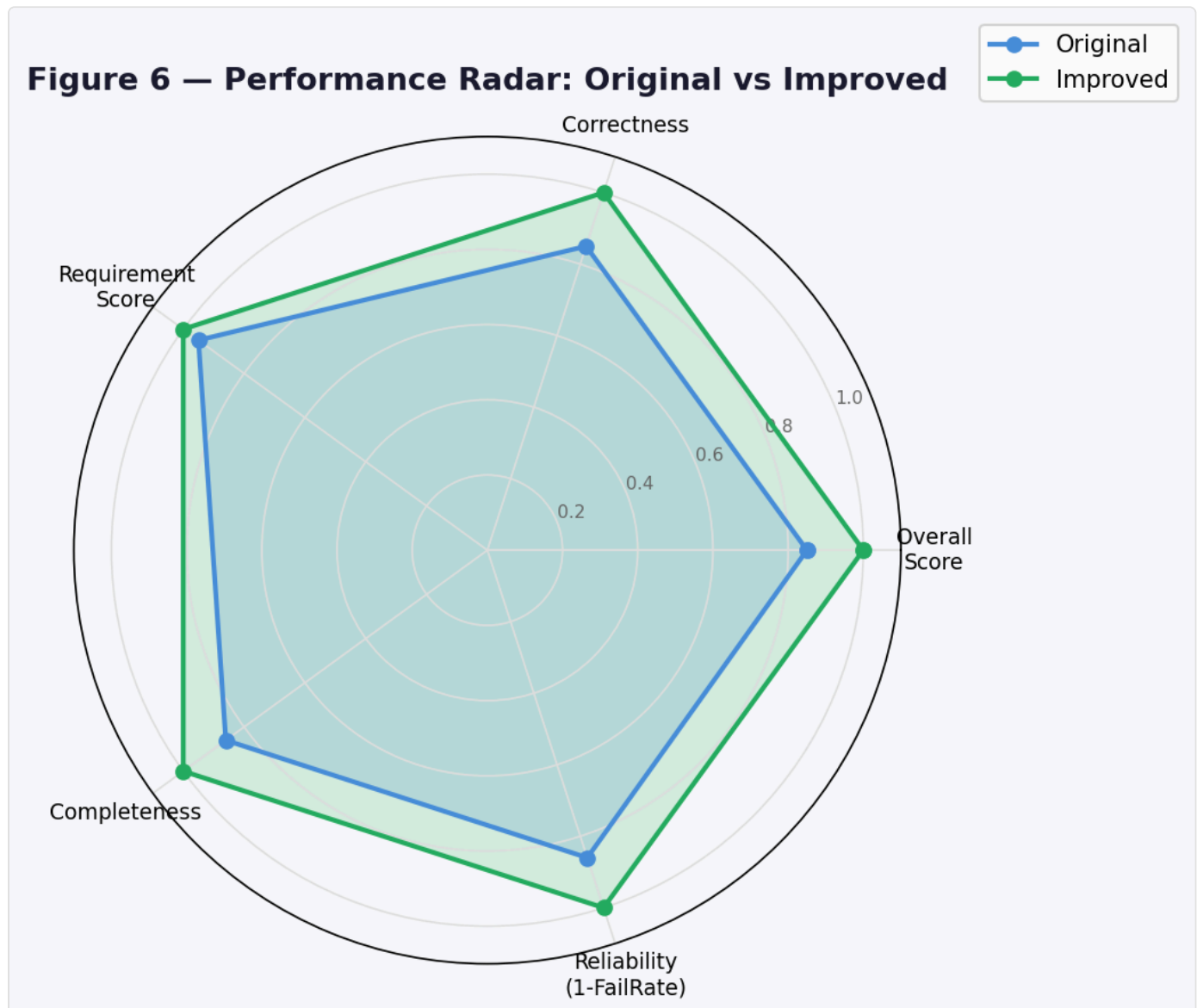


Figure 6 — Performance radar chart. The improved system (green) fully covers the original (blue) on every dimension.

Key Per-Test Results

Case	Original Result	Improved Result	Interpretation
A4-T2 Ambiguous this	Published with topic this and it; score 0.00	Asked for topic clarification; score 1.00	Confidence gate prevents meaningless publish
A4-T4 Jewelry constraints	Requirement score 0.67	Requirement score 1.00	Structured constraints improve image brief coverage
A4-T1, T3, T5, T6, T7	Complete and approved	Complete and approved	Improvements did not break normal routes

Representative Logs

Original ambiguous flow:

```
topic: this and it
qa_status: approved
browser_status: success
```

Improved ambiguous flow:

```
request_brief_verifier low_confidence
task_manager clarification_needed
question: What specific topic or product should the post cover?
```

Improved constraint-aware flow:

```
request_brief_verifier constraints_applied
visual_prompt_engineer validated
requirement_qa approved
```

Part 5 – Discussion

Which Improvements Worked Well?

The structured request brief worked well. It created a clear topic/constraint boundary before generation. The largest gain appeared in the ambiguous prompt: the original system completed a meaningless post, while the improved system treated the same input as insufficient information.

Constraint injection and requirement-aware QA also improved product visual reliability. In the jewelry case, the original result was approved but received only 0.67 visual requirement coverage. The improved result preserved the close-up, material, and count expectations and reached 1.00.

Which Improvements Did Not Add Visible Benefit in Every Case?

The improved system adds extra logs and verifier state even for easy prompts. For T1, T3, T5, T6, and T7 both systems already produced correct deterministic outputs. The extra verification did not change the final score there. The requirement repair feedback loop did not trigger in the deterministic aggregate run. This is not a bug: the pre-generation request brief constraint injection prevented the tested mismatches before QA. The repair path is still needed for real provider outputs, where a text or image model may ignore instructions after prompt generation.

Unexpected Outcomes

Mean latency became lower for the improved system in the deterministic run. This occurs because the improved ambiguous run stops after three logs, while the original baseline continues through writing, media generation, QA, and browser upload. The result supports early stopping as an efficiency strategy for low-confidence requests.

New Problems or Trade-Offs

- More state fields must remain consistent.
- The verifier rules need maintenance as new domains and languages are added.
- Human approval gates can slow publishing if triggered too often.
- Requirement scoring currently checks prompt coverage, not the final aesthetics of a generated image.

The improved design is still better for this workflow because it makes uncertain decisions explicit rather than hiding them behind a completed pipeline.

Part 6 – Reflection: Human + AI Collaboration

Humans should not manually approve every low-risk draft. AI agents can independently route work, draft captions, generate image prompts, validate file existence, and prepare local previews when confidence is high and requirements are clear.

Human oversight is appropriate when:

- Topic confidence is low.
- A post is being published to a real external account.
- Brand, legal, safety, or sensitive product claims are involved.
- QA had to repair a requirement mismatch.
- Generated media must accurately represent a product or person.

The practical design principle is selective oversight. Humans should remain in the loop for high-impact or low-confidence decisions, while agents handle repetitive drafting and validation steps. A reliable agentic system is not one that runs autonomously at all costs. It is one that knows when autonomy is justified and when uncertainty should be exposed to a human.

Responsible Use of AI

AI tools were used to help implement, test, and document the system. The experiment design, metric definitions, interpretation of results, and limitations remain the responsibility of the authors. The deterministic experiment is intentionally separated from real provider behavior so that architectural improvements are not confused with temporary API availability.